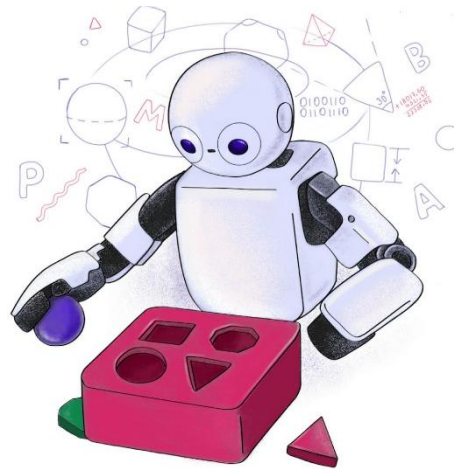
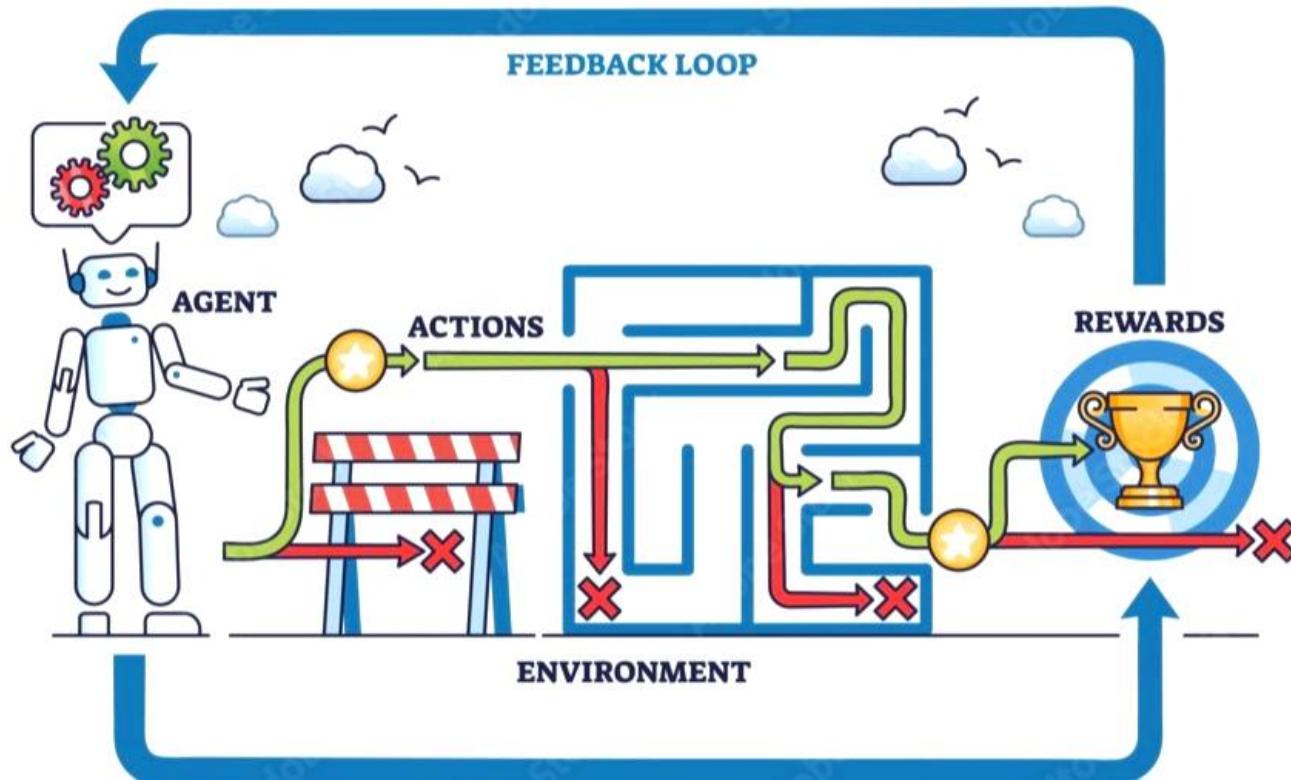


C24 – Inteligência Artificial: *Q-learning*

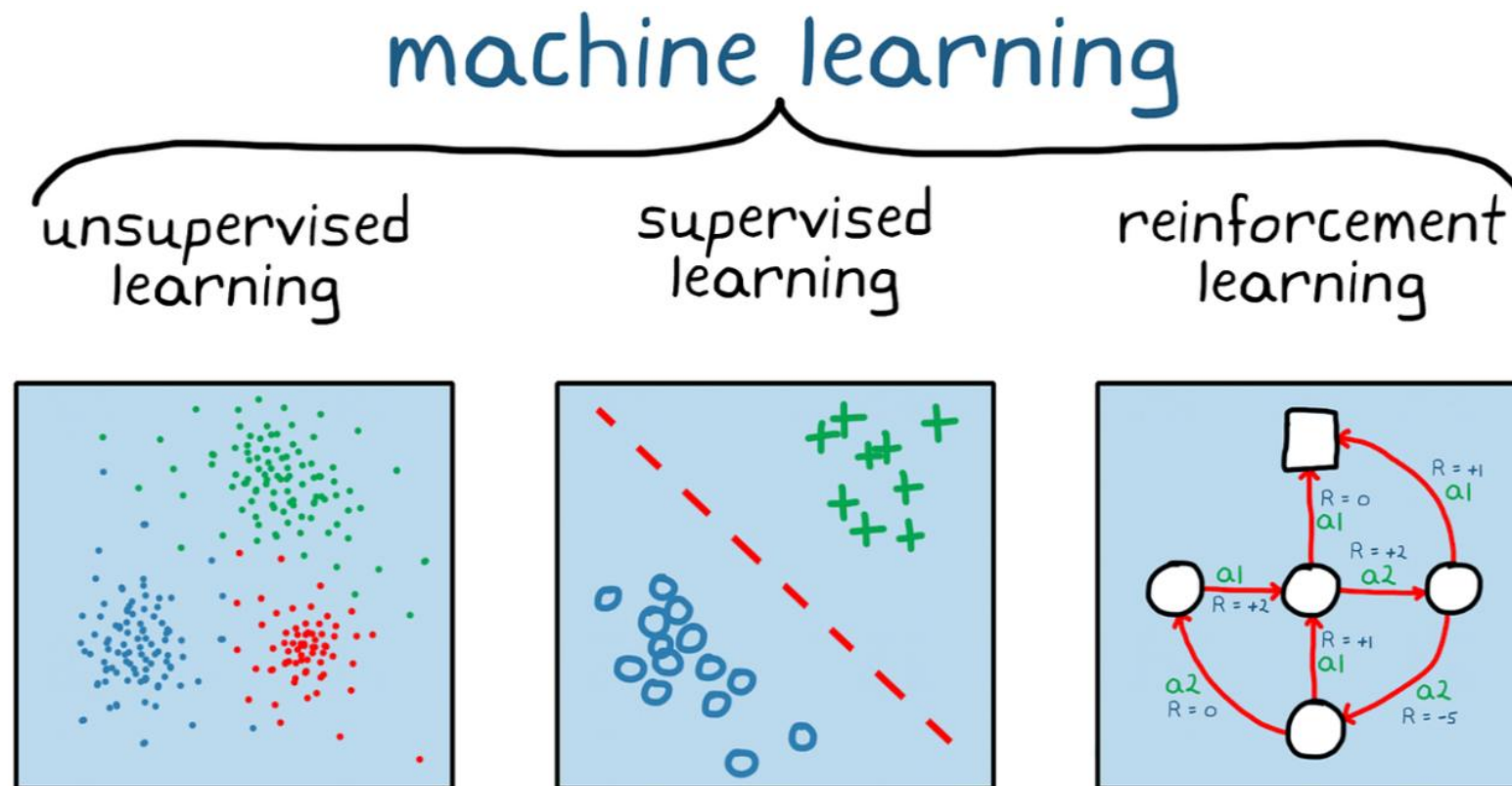




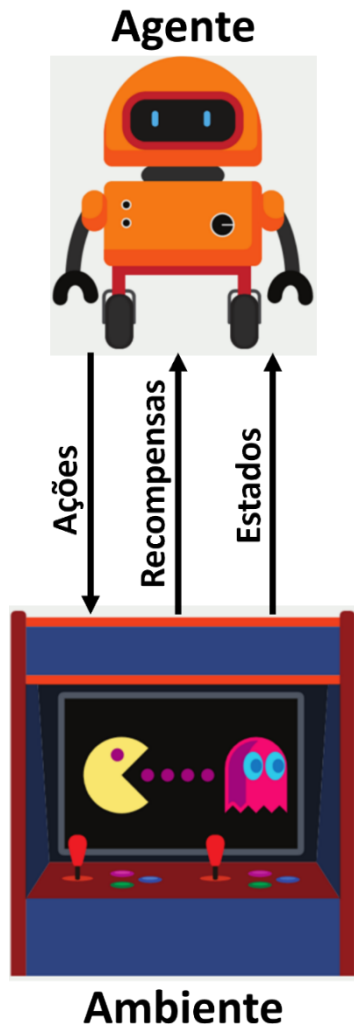
- As **máquinas** podem **aprender** a **tomar decisões ótimas sozinhas sem supervisão** (*dataset*), apenas por **tentativa e erro**.
- Esse é o coração do aprendizado por reforço, em inglês, *reinforcement learning* (RL).
- Nessa aula, aprenderemos o funcionamento do algoritmo mais clássicos da área: o Q-learning.

O que é *reinforcement learning*?

- É um dos paradigmas de aprendizado de *machine learning* focado em como **tomar ações para maximizar as recompensas futuras**.

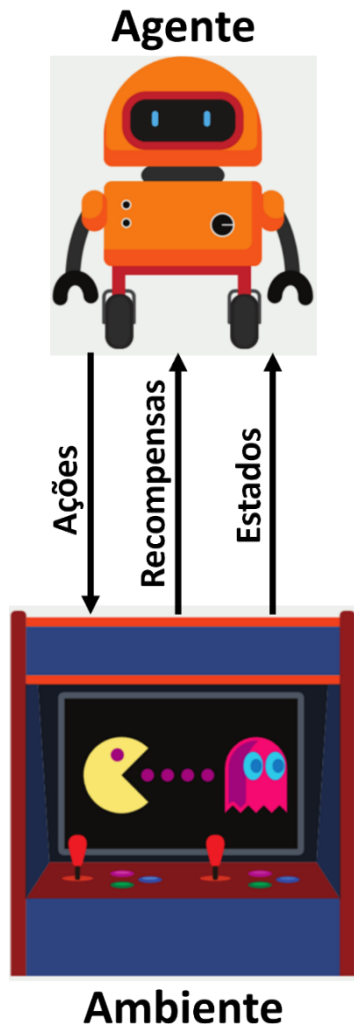


Diferenças chave entre os paradigmas



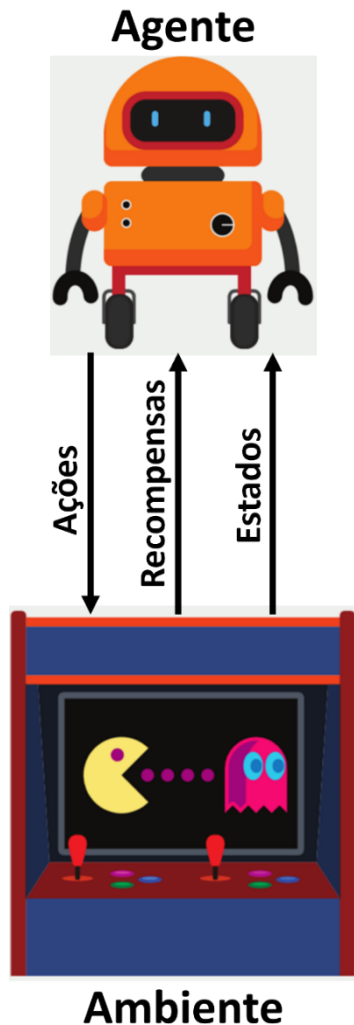
- Ao contrário dos aprendizados supervisionado e não supervisionado, **não existe um *dataset***, seja ele rotulado ou não.
- Não há um *dataset*: o agente **gera seus próprios dados** enquanto **tenta, erra, acerta e aprende**.
 - Nesse contexto, **o agente é o algoritmo de aprendizado por reforço**.
- O aprendizado ocorre por **experiência** através de **interações com um ambiente**.
 - **Ambiente**: uma rede de computadores, um jogo de vídeo game, as ruas de uma cidade, etc.

Diferenças chave entre os paradigmas



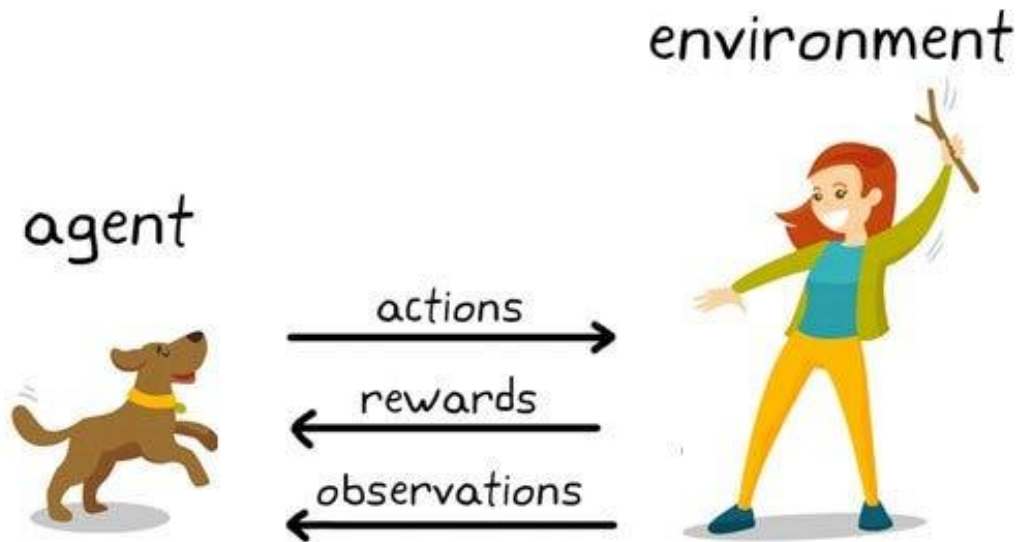
- A **algoritmo aprende** na prática, sendo **premiado ou penalizado pelo ambiente** **por ações tomadas até descobrir a melhor estratégia**, chamada de **política** nesse contexto.
- Existe um sinal de **recompensa ou reforço enviado pelo ambiente**.
 - O ambiente diz ao agente: "Isso foi bom (+10)" ou "Isso foi péssimo (-5)".
- O **reforço é um guia**, **ele não diz qual era a ação certa**, apenas se a que foi tomada foi boa ou ruim.

Diferenças chave entre os paradigmas



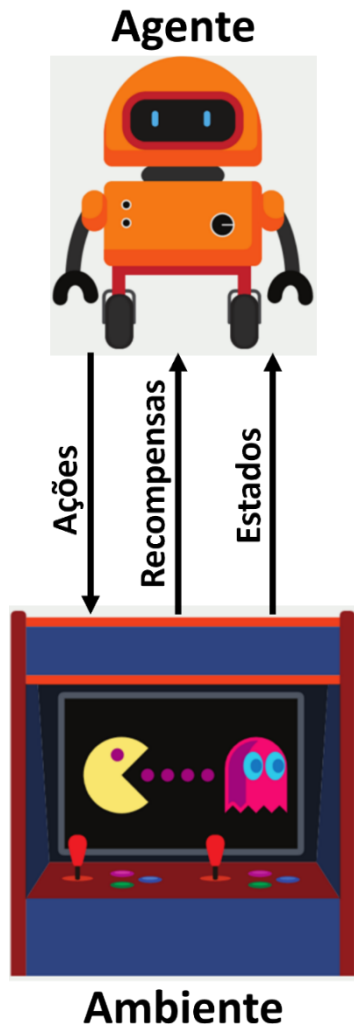
- O aprendizado por reforço **foca na tomada de decisão** para **maximizar as recompensas ao longo do tempo**.
- Ele responde à pergunta:
"Qual sequência de ações eu devo tomar para ganhar o prêmio lá na frente?"

Uma analogia simples



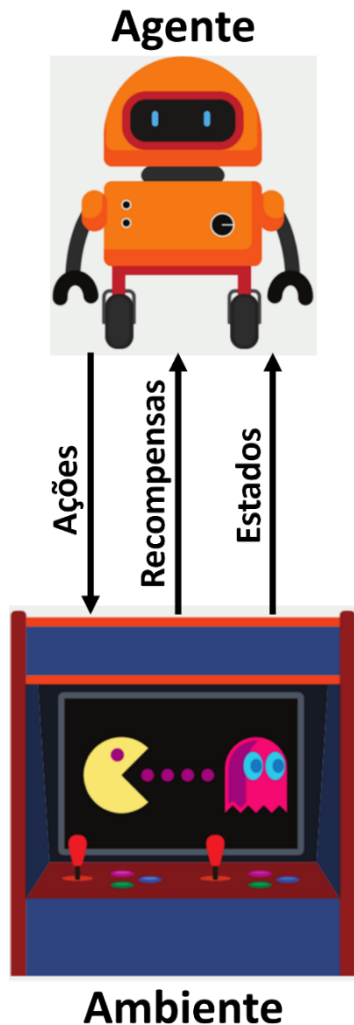
- Aprendizado por reforço é como adestrar um cachorro.
- Você **não diz a ele como se sentar ou pegar** o graveto.
 - Não há dados rotulados ensinando os movimentos musculares.
- Você dá um petisco (recompensa) quando ele senta ou pega o graveto, e ele aprende a associar a ação ao resultado positivo.

Conceitos fundamentais de RL



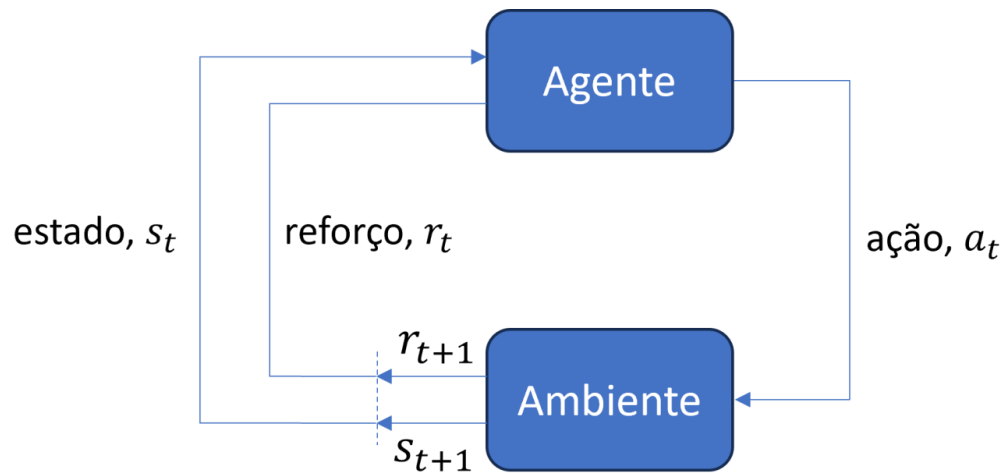
- **Agente:** O tomador de decisões.
 - Exemplo: um robô, um personagem de vídeo *game*, um carro autônomo.
- **Ambiente:** O mundo com o qual o agente interage.
 - Exemplo: um galpão, uma cidade, um tabuleiro de jogo.
- **Estado (s):** Representação da situação atual do ambiente percebida pelo agente.
 - Exemplo: A posição do agente, localização dos objetos/obstáculos em um galpão, cidade ou jogo, velocidade e nível de bateria de um veículo.

Conceitos fundamentais de RL



- **Ação** (a): O que o agente pode fazer para interagir e poder alterar o estado do ambiente.
 - Exemplo: movimentos (esquerda, direita, para cima/baixo), acelerar, frear.
- **Recompensa** (r): O *feedback* (positivo ou negativo) fornecido pelo ambiente após a execução de uma ação.
- **Política** (π): Estratégia utilizada pelo agente para escolher ações em cada estado.
 - É uma função que mapeia o estado atual na melhor ação para aquele estado.

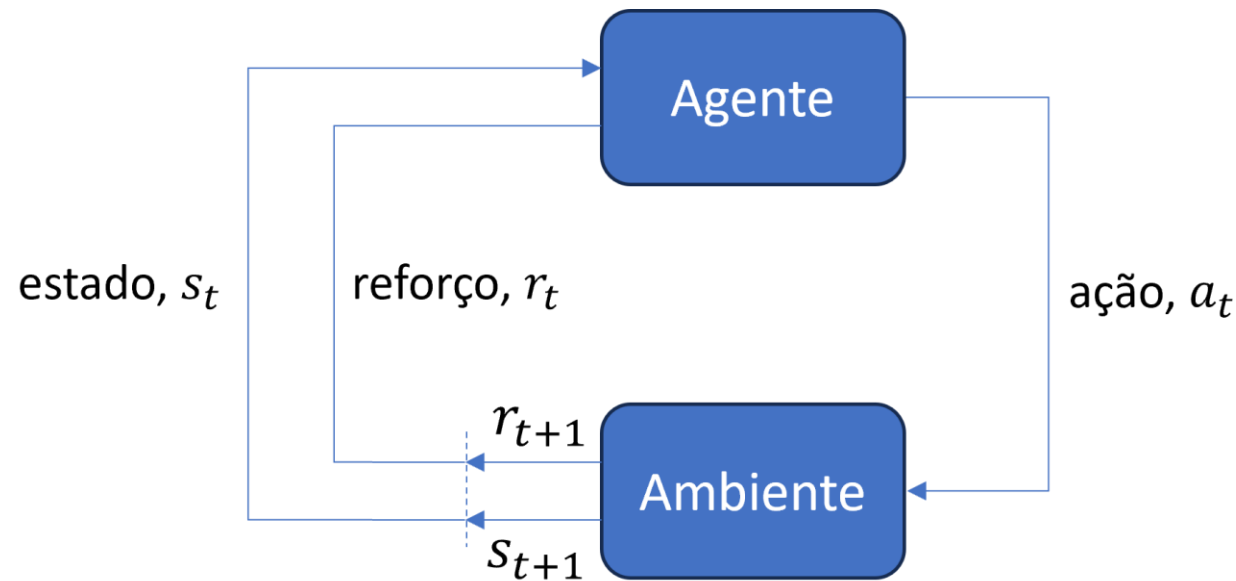
Conceitos fundamentais de RL



- O agente e o ambiente interagem em etapas de tempo discreto: $t, t + 1, t + 2, \dots, T$.
- A cada instante t , o agente
 - observa o estado $s_t \in S$ (conjunto de estados possíveis).
 - com base em s_t , ele seleciona e executa uma ação $a_t \in A$ (conjunto de ações disponíveis).
 - um instante depois, como consequência de sua ação, o agente recebe uma recompensa, $r_{t+1} \in R$ (conjunto de todas as possíveis recompensas).
 - e se encontra em um novo estado, s_{t+1} .

Conceitos fundamentais de RL

- O **objetivo** do agente é **maximizar a soma total de recompensas ao longo do tempo**, não apenas a recompensa imediata.



Algoritmos de RL

- Alguns dos algoritmos de RL mais conhecidos e usados são:
 - ***Q-learning***
 - Um dos algoritmos mais clássicos.
 - **Espaços de ações e estados discretos.**
 - Base de muitos métodos modernos.
 - ***Deep Q-Network (DQN)***
 - Combina *Q-learning* com redes neurais profundas **para grandes conjuntos de ações e estados.**
 - **Espaço de ações discreto e espaço de estados contínuo.**
 - ***DDPG (Deep Deterministic Policy Gradient)***
 - **Extensão do DQN para espaços de ações e estados contínuos.**
- Devido a sua simplicidade e por introduzir de forma clara os conceitos fundamentais de aprendizado por reforço, focaremos no ***Q-learning***, proposto por Christopher Watkins em 1989.

O que é Q-learning?

- Q-learning é um algoritmo **model-free**.
- Em RL, o "**modelo**" é o conhecimento das **regras do ambiente**:
 - As probabilidades de transição do ambiente
$$P(s_{t+1} | s_t, a)$$
 - A função de recompensa
$$R(s_t, a)$$
- O agente de Q-learning **não conhece nem tenta aprender** explicitamente essas regras.

O que é Q-learning?

- O Q-learning aprende uma **função valor-ação** a partir da **interação** com o ambiente:

$$Q(s_t, a).$$

- Ele começa “cego” e por tentativa e erro vai **aprendendo (i.e., atualizando)** a função.
- Por outro lado, os agentes de **algoritmos model-based** **possuem ou aprendem** um modelo do ambiente.
- **Algoritmos model-based** aprendem o modelo do ambiente para **planejar ações antes de executá-las**.

O que significa o "Q"?

Estados	Ações			
	10	2	6	8
	1	9	5	2
	3	6	1	8

- Significa **qualidade**.
- Representa o **quão boa é uma determinada ação em um estado específico**.
- Quem informa a **qualidade** é a **função valor-ação**,
 $Q(s_t, a)$.
- Ela mede o retorno (qualidade) esperado ao executar a ação a no estado s_t .

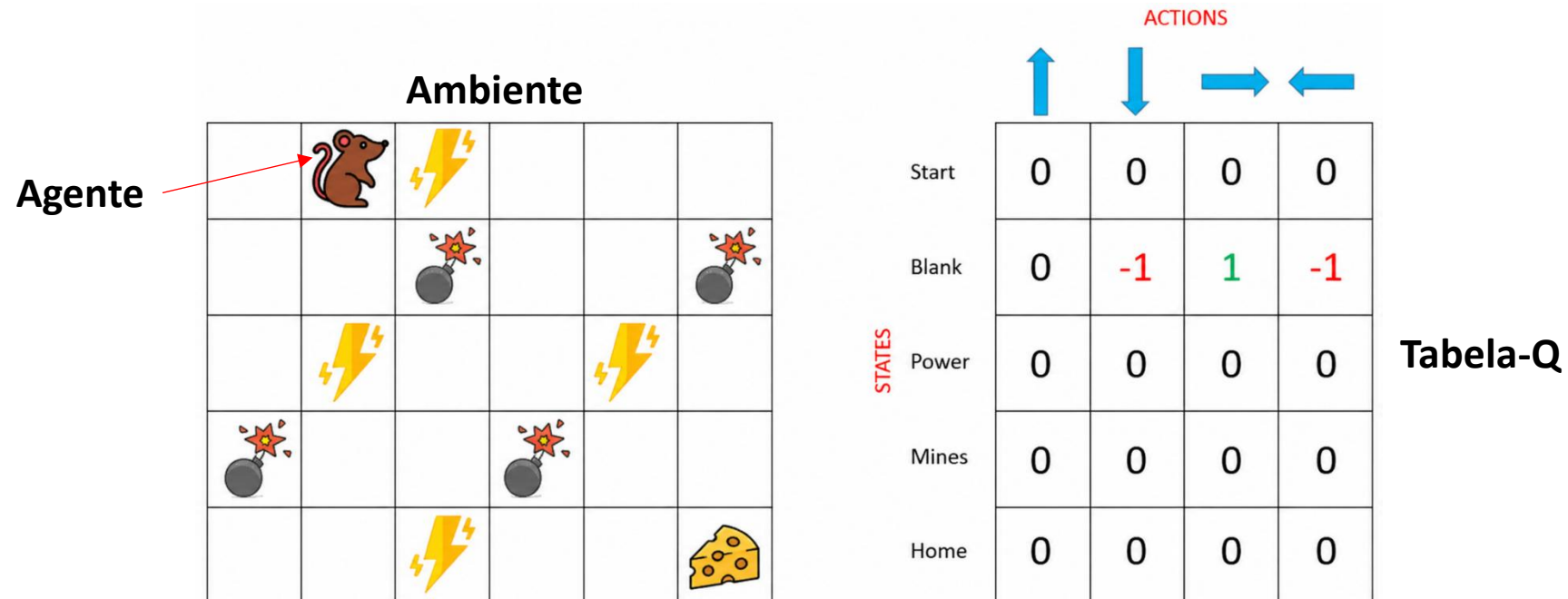
O que significa o "Q"?

Estados	Ações			
	10	2	6	8
	1	9	5	2
	3	6	1	8

- $Q(s_t, a)$ mensura a **recompensa total esperada ao executar uma ação específica partindo de um determinado estado**.
- No *Q-learning*, a função é definida de forma **discreta** (estados e ações) **por uma tabela (ou matriz)**, chamada de **tabela-Q**.
- As linhas são os estados e as colunas são as ações.

O que significa o "Q"?

- Cada entrada da tabela armazena um valor de qualidade para um par estado-ação.
- O **objetivo do algoritmo é preencher a tabela** com os valores que **façam o agente escolher a melhor ação sempre**.



Como o *Q-learning* aprende?

1. Inicialize a tabela-Q com zeros.
2. Repita até o fim do **episódio**:
 - Observe o estado atual, s .
 - Escolha uma ação a usando uma **política (e.g., ϵ -greedy)** baseada em $Q(s, a)$.
 - Execute a ação a , observe o novo estado s' e a recompensa r .
 - **Atualize o valor $Q(s, a)$ na tabela-Q.**
 - Atualize o estado atual: $s \leftarrow s'$.

*Por motivos de brevidade, vamos usar s para representar s_t , s' para representar s_{t+1} e r para representar r_{t+1} .

O 'aprender' acontece na etapa de atualização da tabela-Q, que veremos a seguir.

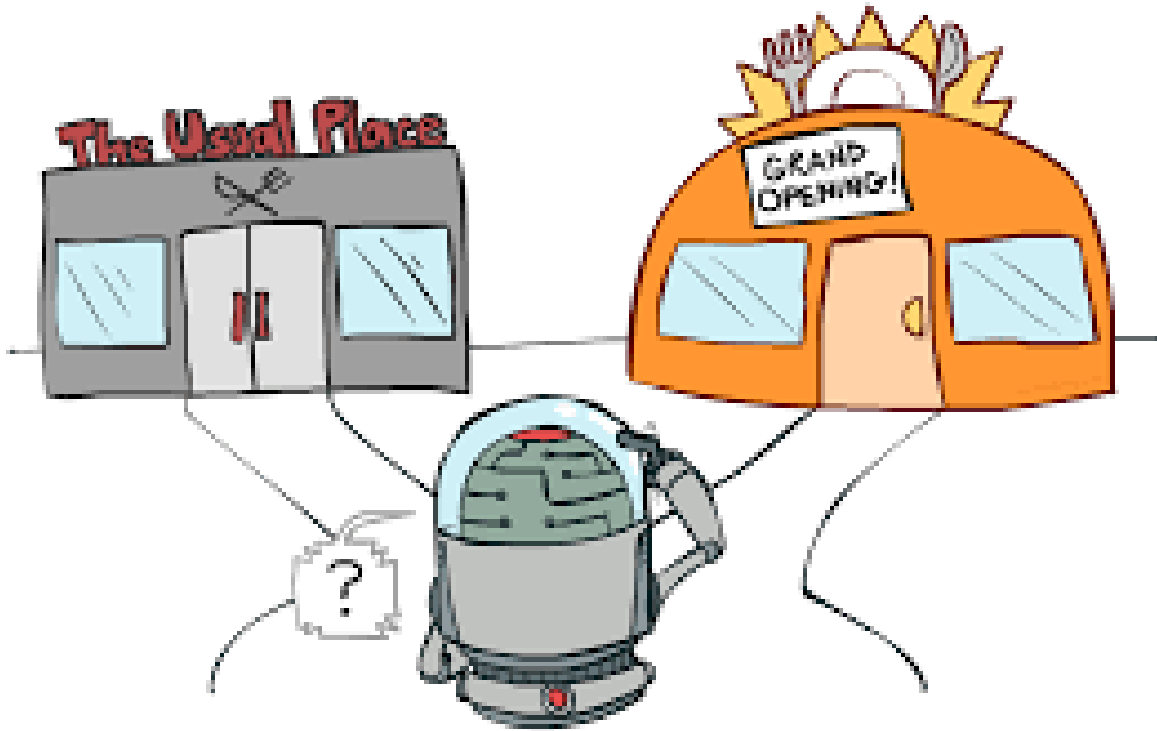
O que é um episódio?

- É uma **sequência completa de interações entre o agente e o ambiente**, que **começa num estado inicial e termina quando um estado terminal é alcançado**.
- O episódio é como uma "partida" de um jogo:
 1. **Início:** O ambiente e o agente são resetados para uma configuração inicial (e.g., o rato aparece no início do mapa).
 2. **Ciclo:** O agente toma ações, recebe recompensas e o ambiente muda de estado sucessivamente.
 3. **Fim:** O episódio se encerra quando um estado terminal é atingido:
 - O agente atinge o **objetivo** (e.g., rato chegar ao queijo).
 - O agente comete um **erro fatal** (e.g., rato encontrar uma bomba).
 - O tempo ou o **limite de passos** se esgota.

O que é um episódio?

- O treinamento do *Q-learning* é feito ao longo de muitos episódios.
- Em cada **novo episódio**, o **agente tenta aprender uma sequência de ações que resulte em uma recompensa total maior do que no anterior**.
- O **treinamento** encerra-se quando
 - a **soma das recompensas obtidas ao longo dos episódios para de aumentar** e "estaciona" em um valor alto por muitos episódios seguidos.
 - Isso significa que o agente já **encontrou uma política ótima**.
 - ou pela **ausência de mudanças significativas nos valores da tabela-Q**.

Exploração vs. exploração (ϵ -greedy)

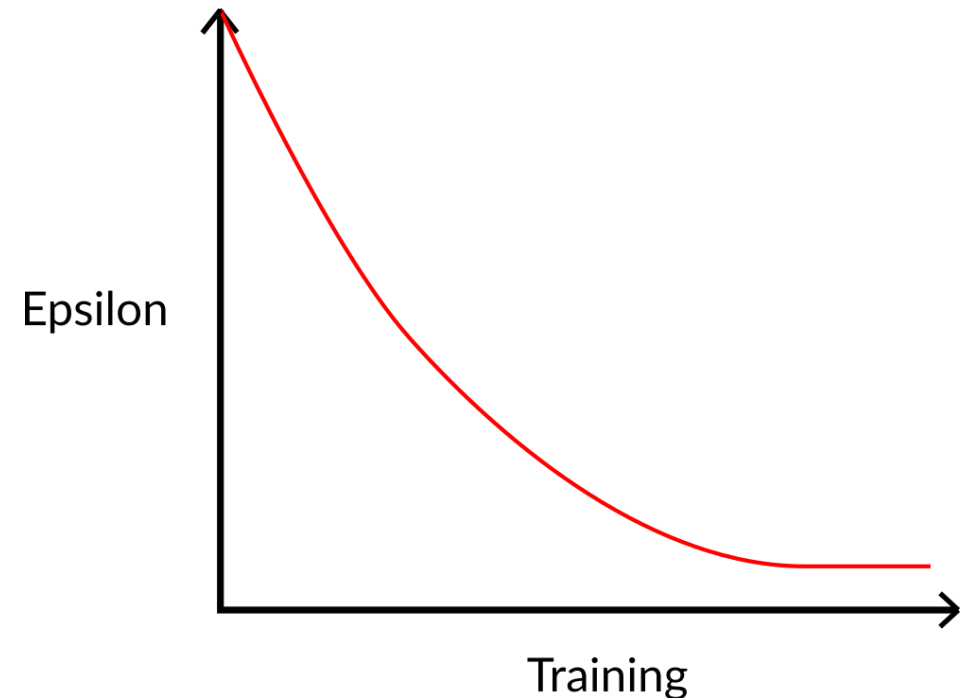


Vocês preferem ir a um restaurante que já conhecem e sabem que é bom ou arriscar e ir a um novo, que pode ser ótimo?

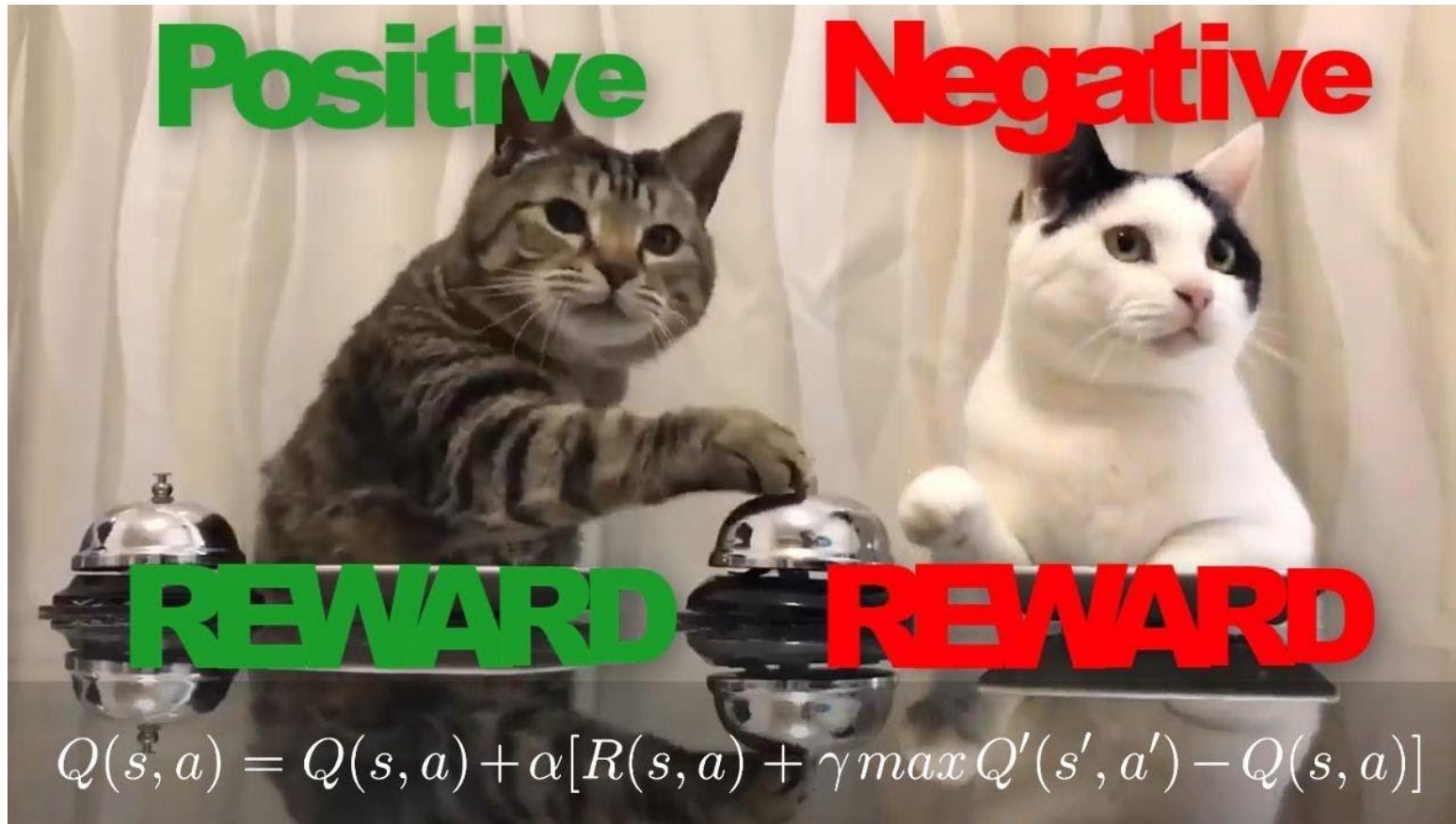
Exploração vs. exploração (ϵ -greedy)

- **Exploração:** Tentar ações aleatórias para descobrir quais podem dar maiores recompensas.
 - Riscos altos, mas possíveis ganhos altos a longo prazo.
- **Exploração:** Escolher a melhor ação conhecida usando a tabela-Q atual.
 - Garantia de recompensa conhecida.
- **A estratégia Epsilon-Greedy (ϵ -greedy) faz um balanço entre exploração e exploração:**
 - Com probabilidade ϵ (e.g., 90%), o agente **explora** (ação aleatória).
 - Com probabilidade $1 - \epsilon$ (e.g., 10%), o agente **explora** (escolhe o valor máximo da tabela Q para o estado atual s).

Exploração vs. exploração (ϵ -greedy)



- Se o **agente não explorar** no começo do treinamento, **ele pode encontrar um caminho medíocre** e ficar preso nele para sempre.
- Para que o agente tenha a chance de aprender uma boa política, a **exploração deve ser agressiva no início**.
- Assim, o ϵ costuma
 - começar em 1.0 (100% de exploração), fazendo com que o agente tome ações aleatórias para descobrir o ambiente
 - e decair gradualmente à medida que o agente aprende (*decay rate*).



Como o agente aprende?

Ou seja, como a tabela-Q é atualizada?

Atualização da tabela-Q: a equação de Bellman

- A tabela-Q é atualizada baseando-se a equação proposta por Richard Bellman:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- $Q(s, a)$: é o valor atual na tabela para o estado, s e a ação, a .
- α : **taxa de aprendizado, varia no intervalo (0, 1]**.
 - Define o **quanto a nova informação atualiza a antiga**.
 - Valor pequeno $\rightarrow$ aprendizado lento.
 - Valor grande $\rightarrow$ atualização agressiva (aprendizado mais rápido, mas pode divergir).
- r : **recompensa**.
 - O que ele acabou de ganhar por tomar a ação, a .
 - Não há restrição de intervalo, mas manter r em intervalos pequenos, como $[-1, 1]$ e $[0, 1]$, ajuda na estabilidade e na velocidade de convergência.

Atualização da tabela-Q: a equação de Bellman

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- γ : **fator de desconto, varia no intervalo [0, 1]**.
 - Determina a **importância das recompensas futuras**.
 - Equilibra a importância entre recompensas imediatas e futuras.
 - Determina o *quão longe no futuro o algoritmo enxerga*.
- $\max_{a'} Q(s', a')$: **é o maior valor Q possível no próximo estado**.
 - É a **estimativa da recompensa futura**.
 - A equação assume que no próximo estado tomaremos a **melhor ação possível**.

Erro de diferença temporal

$$\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{alvo}} - \underbrace{Q(s, a)}_{\text{estimativa atual}}$$

- O termo dentro dos colchetes é chamado de **erro de diferença temporal** (*temporal difference* (TD) *error*).
- É a diferença entre o **alvo de Bellman**, $r + \gamma \max_{a'} Q(s', a')$, e a estimativa atual, $Q(s, a)$.
- Multiplicamos o erro por α para corrigir nossa crença passo a passo (α controla a velocidade de aprendizado).
- **A cada iteração, $Q(s, a)$ se aproxima do alvo**, i.e., da recompensa imediata, r , mais o valor futuro ótimo descontado, $\gamma \max_{a'} Q(s', a')$.

A equação de Bellman

- A equação de Bellman diz basicamente:

“O valor de uma ação = recompensa imediata + maior valor futuro possível descontado”

- Ou seja

$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a').$$

- A equação de Bellman pode ser interpretada como:

Novo valor = Valor antigo + α × erro

- Ou seja, **ajusta-se a estimativa da função $Q(s, a)$ na direção do valor alvo.**

A influência do fator de desconto, γ

- $\gamma \leftarrow 0$: o agente só liga para recompensas imediatas.
 - Foca no curto prazo, **recompensas futuras não importam**.
 - O aprendizado é mais rápido.
 - Usado em sistemas de controle em tempo real, **onde o que importa é o agora**.
 - Por exemplo: controle de acesso ao meio em redes de telecom (transmitir agora ou esperar), controle de marca-passos, *day trading* (comprar ou vender ações a cada minuto).
- $\gamma \rightarrow 1$: o agente busca recompensas a longo prazo.
 - Ele **valoriza recompensas que ocorrerão muito tempo depois**.
 - O treinamento é mais lento.
 - Usado em navegação autônoma ou robótica, jogos de estratégia, planejamento onde ações **iniciais afetam o resultado final e a recompensa só vem muito tempo depois ou no final**.
 - Por exemplo: robô indo até um destino distante, *pac man*, xadrez, decidir trajetórias eficientes para veículos autônomos (planejamento de rota).

Off-policy

- Outra característica do *Q-learning* é que ele é um **algoritmo de aprendizado *off-policy*** (fora da política).
- Isso significa que o agente **aprende uma política diferente** daquela usada **para explorar o ambiente**.
- O algoritmo **executa ações exploratórias**, **mas aprende como se estivesse sempre escolhendo a melhor ação possível**.

Off-policy

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- O agente **pode ter executado qualquer ação** para chegar em s' .
- Porém, ele atualiza $Q(s, a)$ **assumindo que o agente sempre escolhe a ação ótima futura, independentemente da ação realmente executada.**
- Mesmo que o algoritmo tenha explorado (ação aleatória), ele **aprende como se tivesse agido de forma ótima.**
- Já algoritmos *on-policy* usam a **ação realmente escolhida e executada**, a' , para atualizar $Q(s, a)$

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)].$$

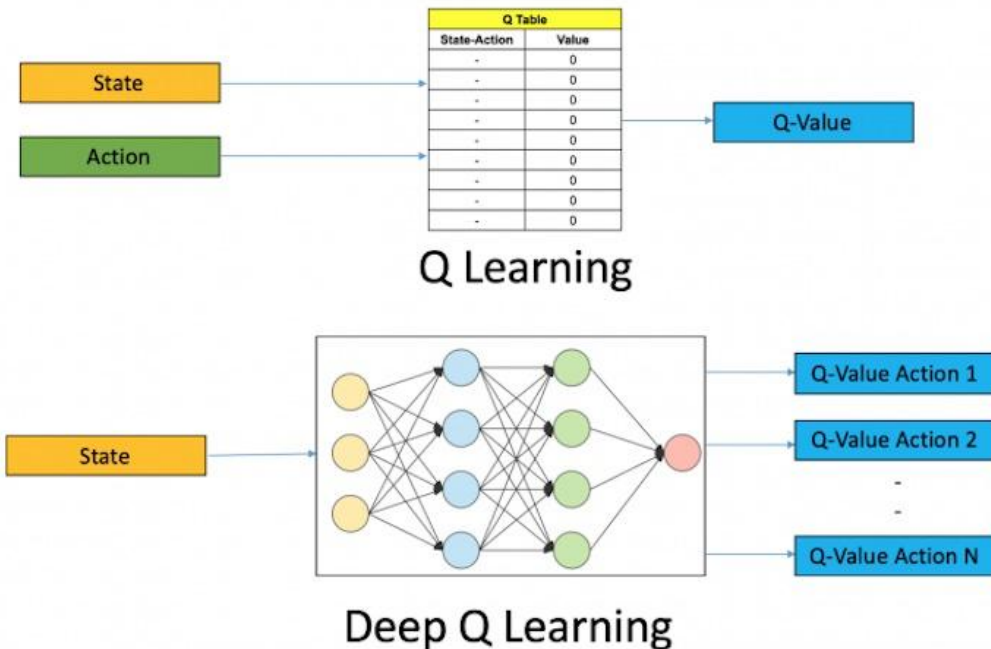
A maldição da dimensionalidade

- O *Q-learning* sofre com a **maldição da dimensionalidade**.
- Ele requer **armazenar um valor $Q(s, a)$ para cada par estado-ação**.
- Em problemas simples isso é viável.
- Entretanto, em problemas reais, o **espaço de estados cresce exponencialmente**.
 - Xadrez: são entre 10^{43} e 10^{47} estados possíveis.
 - Carro autônomo: imagens com milhões de *pixels* geram um espaço de estados extremamente grande.
- Nesses casos, **armazenar ou percorrer a tabela-Q torna-se computacionalmente inviável**.

A maldição da dimensionalidade

- Para problemas com **poucas ações e estados discretos**, a **tabela é ótima**.
- Porém, em vários casos práticos os **estados são contínuos**.
- Por exemplo, robótica, a **velocidade e o ângulo da articulação dos braços mecânicos são valores contínuos**.
- Isso faz com que o **número estados possíveis seja infinito**.
- Assim, se o espaço de estados é **grande e/ou contínuo** a tabela-Q se torna inviável.
- Precisamos **substituir a tabela por um aproximador de funções**, como uma rede neural.

Trocando a tabela-Q por redes neurais



- **A solução:** Em vez de usar uma tabela para representar $Q(s, a)$, usamos uma **rede neural para aproximá-la**.
- A **rede aprende a generalizar estados**: o agente **não precisa visitar todos os estados**.
- **Entrada:** O estado (e.g., a imagem da câmera).
 - Agora, o estado pode ser composto por valores contínuos.
- **Saída:** Os valores Q para todas as ações possíveis.
- Esse algoritmo é chamado de *deep Q-network* (DQN).

Onde RL é usado?



- **Robótica e manufatura:** Ensinar robôs a andar, agarrar objetos ou equilibrar cargas.
- **Otimização de redes e sistemas:** Roteamento de pacotes dinâmico para evitar gargalos na rede.
- **Sistemas de recomendação:** Otimizar qual vídeo ou anúncio mostrar para você na internet com base no engajamento a longo prazo.
- **Controle industrial:** Ajuste fino de caldeiras e refrigeração em *data centers* para reduzir consumo de energia.

Exemplo numérico

Exemplo numérico

início	neutro
bomba	meta

- Vamos supor um tabuleiro que possui 4 **estados**:
 - Início (S): estado inicial de cada episódio, recompensa 0, não terminal.
 - Célula neutra (N): recompensa 0, não terminal.
 - Bomba (B): recompensa -1, **terminal**.
 - Meta (G): recompensa +1, **terminal**.
- **Ações**: cima, baixo, esquerda, direita.
- **Transições com bordas**: se uma ação levar para fora do tabuleiro, o agente **permanece na mesma célula**. A recompensa é a da célula onde ele permanece.
- **Dinâmica dos terminais**: ao atingir a Meta ou a Bomba, o episódio termina imediatamente. Não há ações seguintes nesses estados.

Exemplo numérico

início	neutro
bomba	meta

Parâmetros escolhidos para o exemplo:

- Taxa de aprendizado $\alpha = 0.5$.
- Fator de desconto $\gamma = 0.9$.
- **Política de exploração:** ε -greedy com $\varepsilon = 0.5$ (50% de chance de ação aleatória, 50% de ação gulosa).
- Iniciaremos o exemplo com esse ε e depois consideraremos uma fase puramente gulosa para observar a política aprendida.
- **Inicialização:** todos os $Q(s, a) = 0$.
- Bomba e Meta são terminais: nunca selecionamos ações neles.

Exemplo numérico

início	neutro
bomba	meta

Tabela Q inicial

Estado/Ação	Cima	Direita	Baixo	Esquerda
Início	0	0	0	0
Neutro	0	0	0	0
Bomba	0	0	0	0
Meta	0	0	0	0

Agora vamos simular alguns episódios, mostrando a cada passo, o cálculo da atualização e a evolução da tabela.

Exemplo numérico

Episódio 1

• Passo 1

início	neutro
bomba	meta

- Estado atual: **início**.
- Como todos os valores de $Q(s, a) = 0$, suponha que, por exploração, o agente selecionou a ação: **baixo**.
- Transição: **início** + **baixo** → **bomba**.
- Recompensa: $r = -1$, estado seguinte $s' = \mathbf{bomba}$ (terminal).
- $\max_{a'} Q(\mathbf{bomba}, a') = 0$ (terminal, Q não se altera).
- Atualização de $Q(\mathbf{início}, \mathbf{baixo})$:
$$Q(\mathbf{início}, \mathbf{baixo}) = 0 + 0.5[-1 + 0.9 \times 0 - 0] = -0.5$$
- Episódio terminado.

Exemplo numérico

início	neutro
bomba	meta

Tabela Q após episódio 1

Estado/Ação	Cima	Direita	Baixo	Esquerda
Início	0	0	-0.5	0
Neutro	0	0	0	0
Bomba	0	0	0	0
Meta	0	0	0	0

Exemplo numérico

Episódio 2

• Passo 1

início	neutro
bomba	meta

- Ações com maior Q em **início**: **cima, direita e esquerda**.
- A política poderia escolher qualquer uma delas, mas vamos supor que a escolha foi aleatória (exploração): **direita**.
- Transição: **início + direita** → **neutro**.
- Recompensa: $r = 0$, estado seguinte $s' = \mathbf{neutro}$ (não terminal).
- $\max_{a'} Q(\mathbf{neutro}, a') = 0$ (todos zero).
- Atualização de $Q(\mathbf{início}, \mathbf{direita})$:
$$Q(\mathbf{início}, \mathbf{direita}) = 0 + 0.5[0 + 0.9 \times 0 - 0] = 0$$
- O valor permanece 0.

Exemplo numérico

Episódio 2

• Passo 2

início	neutro
bomba	meta

- Agora o agente está em **neutro**.
- Todas as ações em **neutro** com $Q = 0$.
- Suponha que, por exploração, o agente escolheu **baixo**.
- Transição: **neutro** + **baixo** → **meta**.
- Recompensa: $r = +1$, estado seguinte $s' = \mathbf{meta}$ (terminal).
- $\max_{a'} Q(\mathbf{meta}, a') = 0$ (todos zero).
- Atualização de $Q(\mathbf{neutro}, \mathbf{baixo})$:
$$Q(\mathbf{neutro}, \mathbf{baixo}) = 0 + 0.5[1 + 0.9 \times 0 - 0] = 0.5$$
- Episódio terminado.

Exemplo numérico

início	neutro
bomba	meta

Tabela Q após episódio 2

Estado/Ação	Cima	Direita	Baixo	Esquerda
Início	0	0	-0.5	0
Neutro	0	0	0.5	0
Bomba	0	0	0	0
Meta	0	0	0	0

Exemplo numérico

Episódio 3

• Passo 1

início	neutro
bomba	meta

- Ações com maior Q em **início**: **cima, direita e esquerda**.
- Podemos ter exploração ou exploração. Suponha que a escolha foi gulosa (exploração): **direita**.
- Transição: **início + direita** → **neutro**.
- Recompensa: $r = 0$, estado seguinte $s' = \mathbf{neutro}$ (não terminal).
- $\max_{a'} Q(\mathbf{neutro}, a') = 0.5$ (ação **baixo** no estado **neutro**).
- Atualização:
$$Q(\mathbf{início}, \mathbf{direita}) = 0 + 0.5[0 + 0.9 \times 0.5 - 0] = 0.225$$

Exemplo numérico

Episódio 3

• Passo 2

início	neutro
bomba	meta

- Agora o agente está no estado **neutro**.
- Suponha que, por exploração, o agente escolheu **baixo**.
- Transição: **neutro** + **baixo** → **meta**.
- Recompensa: $r = +1$, estado seguinte $s' = \mathbf{meta}$ (terminal).
- $\max_{a'} Q(\mathbf{meta}, a') = 0$ (todos zero).
- Atualização:
$$Q(\mathbf{neutro}, \mathbf{baixo}) = 0.5 + 0.5[1 + 0.9 \times 0 - 0.5] = 0.75$$
- Episódio terminado.

Exemplo numérico

início	neutro
bomba	meta

Tabela Q após episódio 3

Estado/Ação	Cima	Direita	Baixo	Esquerda
Início	0	0.225	-0.5	0
Neutro	0	0	0.75	0
Bomba	0	0	0	0
Meta	0	0	0	0

Exemplo numérico

Episódio 4

• Passo 1

início	neutro
bomba	meta

- Ação com maior Q em **início**: **direita**.
- Suponha que, por exploração, o agente escolheu **direita**.
- Transição: **início** + **direita** → **neutro**.
- Recompensa: $r = 0$, estado seguinte $s' = \mathbf{neutro}$ (não terminal).
- $\max_{a'} Q(\mathbf{neutro}, a') = 0.75$.
- Atualização:
$$Q(\mathbf{início}, \mathbf{direita}) = 0.225 + 0.5[0 + 0.9 \times 0.75 - 0.225] = 0.45$$

Exemplo numérico

Episódio 4

• Passo 2

início	neutro
bomba	meta

- Agora o agente está no estado **neutro**.
- Suponha que, por exploração, o agente escolheu **baixo**.
- Transição: **neutro** + **baixo** → **meta**.
- Recompensa: $r = +1$, estado seguinte $s' = \mathbf{meta}$ (terminal).
- $\max_{a'} Q(\mathbf{meta}, a') = 0$ (todos zero).
- Atualização:
$$Q(\mathbf{neutro}, \mathbf{baixo}) = 0.75 + 0.5[1 + 0 - 0.75] = 0.875$$
- Episódio terminado.

Exemplo numérico

início	neutro
bomba	meta

Tabela Q após episódio 4

Estado/Ação	Cima	Direita	Baixo	Esquerda
Início	0	0.45	-0.5	0
Neutro	0	0	0.875	0
Bomba	0	0	0	0
Meta	0	0	0	0

Exemplo numérico

Episódio 5

• Passo 1

início	neutro
bomba	meta

- Ação com maior Q em **início**: **direita**.
- Suponha que, por exploração, o agente escolheu **cima**.
- Transição: **início** + **cima** → **início**.
- Recompensa: $r = 0$, estado seguinte $s' = \mathbf{início}$ (não terminal).
- $\max_{a'} Q(\mathbf{início}, a') = 0.45$ (direita).
- Atualização:
$$Q(\mathbf{início}, \mathbf{cima}) = 0 + 0.5[0 + 0.9 \times 0.45 - 0] = 0.2025$$
- O agente aprende que mesmo uma ação que não sai do lugar pode ter valor, pois do mesmo estado inicial pode-se depois tomar a ação ótima.

Exemplo numérico

Episódio 5

• Passo 2

início	neutro
bomba	meta

- Ação com maior Q em **início**: **direita**.
- Suponha que, por exploração, o agente escolheu **baixo**.
- Transição: **início** + **baixo** $\rightarrow$ **bomba**.
- Recompensa: $r = -1$, estado seguinte $s' =$ **bomba** (terminal).
- $\max_{a'} Q(\mathbf{bomba}, a') = 0$.
- Atualização:
$$Q(\mathbf{início}, \mathbf{baixo}) = -0.5 + 0.5[-1 + 0.9 \times 0 - (-0.5)] = -0.75$$
- Episódio terminado.

Exemplo numérico

Tabela Q após episódio 5

início	neutro
bomba	meta

Estado/Ação	Cima	Direita	Baixo	Esquerda
Início	0.2025	0.45	-0.75	0
Neutro	0	0	0.875	0
Bomba	0	0	0	0
Meta	0	0	0	0

Exemplo numérico

Episódio 6

• Passo 1

início	neutro
bomba	meta

- O agente está em **início**.
- Suponha que, por exploração, o agente escolheu **direita**.
- Transição: **início** + **direita** → **neutro**.
- Recompensa: $r = 0$, estado seguinte $s' = \mathbf{neutro}$ (não terminal).
- $\max_{a'} Q(\mathbf{neutro}, a') = 0.875$.
- Atualização:
$$Q(\mathbf{início}, \mathbf{direita}) = 0.45 + 0.5[0 + 0.9 \times 0.875 - 0.45] = 0.61875$$

Exemplo numérico

Episódio 6

- **Passo 2**

início	neutro
bomba	meta

- Agora o agente está em **neutro**.
- Suponha que, por exploração, o agente escolheu **baixo**.
- Transição: **neutro** + **baixo** → **meta**.
- Recompensa: $r = +1$, estado seguinte $s' = \mathbf{meta}$ (terminal).
- $\max_{a'} Q(\mathbf{meta}, a') = 0$ (todos zero).
- Atualização:
$$Q(\mathbf{neutro}, \mathbf{baixo}) = 0.875 + 0.5[1 - 0.875] = 0.9375$$
- Episódio terminado.

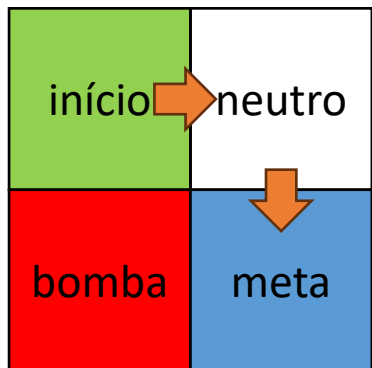
Exemplo numérico

início	neutro
bomba	meta

Tabela Q após episódio 6

Estado/Ação	Cima	Direita	Baixo	Esquerda
Início	0.2025	0.61875	-0.75	0
Neutro	0	0	0.9375	0
Bomba	0	0	0	0
Meta	0	0	0	0

Exemplo numérico



- A convergência final (após muitas visitas) leva aos valores ótimos:
 - $Q^*(\text{início}, \text{direita}) = 0 + \gamma \times 1 = 0.9$
 - $Q^*(\text{neutro}, \text{baixo}) = 1 + \gamma \times 0 = 1.0$
 - $Q^*(\text{início}, \text{cima}) = 0 + \gamma \times 0.9 = 0.81$
 - $Q^*(\text{início}, \text{esquerda}) = 0 + \gamma \times 0.9 = 0.81$ (borda mantém em **início**)
 - $Q^*(\text{neutro}, \text{esquerda}) = 0 + \gamma \times 0.9 = 0.81$ (vai para **início**)
 - $Q^*(\text{início}, \text{baixo}) = -1.0$ (**bomba**, terminal)
- Após muitos episódios, se tornarmos a política completamente gulosa ($\epsilon = 0$), o agente seguirá:
 - Em **início**: escolher **direita** ($Q = 0.9$, a maior)
 - Em **neutro**: escolher **baixo** ($Q = 1.0$)
- Esse caminho leva diretamente à **meta** sem pisar na **bomba**, maximizando a recompensa acumulada descontada.
- **Exemplo:** [q_learning_grid_2x2.ipynb](#)

Exemplo

- [Exemplo: q-learning.ipynb](#)



Lista de exercícios

[Lista #14 – Q-learning](#)

Perguntas?

Obrigado!

Figuras

